

Statistical Optimization: Lecture 6

Ordinary Least Squares and Gradient Descent

Zijian Guo

Zhejiang University
Center for Data Science

March 25, 2026

Least-squares setup

Given data $\{(x_i, y_i)\}_{i=1}^n$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, consider the linear predictor

$$f_{\theta}(x) = x^{\top} \theta.$$

Then the empirical least-squares objective is

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - x_i^{\top} \theta)^2.$$

Let

$$y = (y_1, \dots, y_n)^{\top} \in \mathbb{R}^n, \quad X = \begin{pmatrix} x_1^{\top} \\ \vdots \\ x_n^{\top} \end{pmatrix} \in \mathbb{R}^{n \times d}.$$

Then

$$\widehat{R}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2.$$

Closed-form solution: normal equations

Expand the objective:

$$\widehat{R}(\theta) = \frac{1}{n} (\|y\|_2^2 - 2\theta^\top X^\top y + \theta^\top X^\top X \theta).$$

Hence

$$\nabla \widehat{R}(\theta) = \frac{2}{n} (X^\top X \theta - X^\top y).$$

Setting the gradient to zero gives the **normal equations**

$$X^\top X \widehat{\theta} = X^\top y.$$

If $X^\top X$ is invertible, we have

$$\widehat{\theta} = \left(\frac{1}{n} X^\top X \right)^{-1} \left(\frac{1}{n} X^\top y \right).$$

Orthogonal projection

Let $V \subset \mathbb{R}^n$ be a linear subspace, and let $y \in \mathbb{R}^n$.

A vector $\bar{y} \in V$ is called the **orthogonal projection** of y onto V if

$$y - \bar{y} \perp V.$$

That is,

$$(y - \bar{y})^\top v = 0, \quad \forall v \in V.$$

Interpretation:

- $\bar{y}$ lies in the subspace V ,
- the error $y - \bar{y}$ is orthogonal to the whole subspace,
- so $\bar{y}$ is the point in V that best matches y .

Geometric interpretation of OLS (1/2)

Proposition. The vector of predictions

$$X\hat{\theta} = X(X^\top X)^{-1}X^\top y$$

is the orthogonal projection of $y \in \mathbb{R}^n$ onto $\text{im}(X) \subset \mathbb{R}^n$, the column space of X . Here $\text{im}(X) := \{Xa : a \in \mathbb{R}^d\}$.

Proof.

Let $p := X(X^\top X)^{-1}X^\top y$. Then clearly $p \in \text{im}(X)$. For any $a \in \mathbb{R}^d$, we have

$$a^\top X^\top (y - p) = a^\top X^\top \left(y - X(X^\top X)^{-1}X^\top y \right) = 0.$$

Hence

$$(Xa)^\top (y - p) = 0, \quad \forall a \in \mathbb{R}^d.$$

So $y - p$ is orthogonal to every vector in $\text{im}(X)$.

Therefore $p = X\hat{\theta}$ is the orthogonal projection of y onto $\text{im}(X)$. □

Geometric interpretation of OLS (2/2)

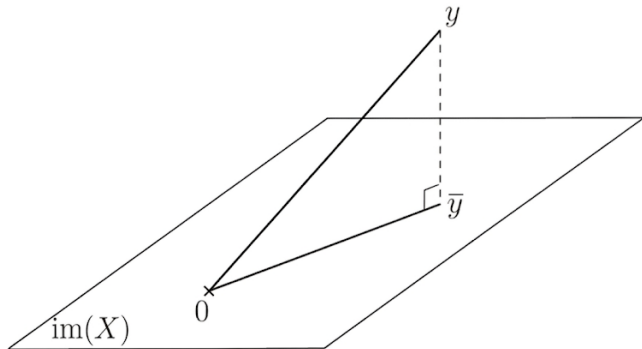


Figure: A geometric interpretation of OLS

Extension to feature maps

We can extend linear regression to nonlinear features by introducing a feature map

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p.$$

Then we still use a linear model in the transformed feature space:

$$f_{\theta}(x) = \phi(x)^{\top} \theta.$$

Define the feature matrix

$$\Phi = \begin{pmatrix} \phi(x_1)^{\top} \\ \vdots \\ \phi(x_n)^{\top} \end{pmatrix} \in \mathbb{R}^{n \times p}.$$

Then the least-squares problem becomes

$$\min_{\theta \in \mathbb{R}^p} \frac{1}{n} \|y - \Phi \theta\|_2^2.$$

Outline

Least Squares and QR decomposition

Least Squares via First Order Method

Least Squares via Second Order Method

QR decomposition: Full Column Rank

$$\begin{array}{ccc} \boxed{\begin{array}{c} X \\ n \times d \end{array}} & = & \boxed{\begin{array}{c} Q \\ n \times d \end{array}} \quad \boxed{\begin{array}{c} R \\ d \times d \end{array}} \end{array}$$

We conduct the QR decomposition as

$$X = QR,$$

where $R \in \mathbb{R}^{d \times d}$ is upper triangular and invertible and

$$Q \in \mathbb{R}^{n \times d}, \quad Q^\top Q = I_d.$$

Applying QR to least squares

Then

$$X^T X = (QR)^T (QR) = R^T Q^T QR = R^T R,$$

and

$$X^T y = (QR)^T y = R^T Q^T y.$$

Recall the normal equation

$$X^T X \hat{\theta} = X^T y.$$

So the normal equation becomes

$$R^T R \hat{\theta} = R^T Q^T y.$$

Since R is invertible, we obtain

$$R \hat{\theta} = Q^T y.$$

So least squares is reduced to solving an upper-triangular system.

Solving OLS with QR in Python

```
import numpy as np

# Input:
# X : (n, d) design matrix
# y : (n,) response vector

# Step 1: QR factorization
Q, R = np.linalg.qr(X, mode='reduced') #thin QR

# Step 2: project y onto the orthonormal basis
b = Q.T @ y

# Step 3: solve the upper-triangular system R theta = b
theta = np.linalg.solve(R, b)
```

This returns the least-squares solution $\hat{\theta}$.

Outline

Least Squares and QR decomposition

Least Squares via First Order Method

Least Squares via Second Order Method

Gradient Descent

We consider the optimization problem

$$\min_{\theta \in \mathbb{R}^d} F(\theta).$$

For a small step $v \in \mathbb{R}^d$, the first-order Taylor expansion gives

$$F(\theta + v) \approx F(\theta) + \nabla F(\theta)^\top v.$$

So to decrease F , we want to choose v such that

$$\nabla F(\theta)^\top v < 0.$$

And the steepest descent direction is $-\nabla F(\theta)$.

Gradient descent (GD) starts from an initial point $\theta_0 \in \mathbb{R}^d$, and updates by

$$\theta_t = \theta_{t-1} - \gamma_t \nabla F(\theta_{t-1}), \quad t \geq 1,$$

where $(\gamma_t)_{t \geq 1}$ is a sequence of step sizes.

Gradient Descent for OLS

For OLS, the objective function for $X \in \mathbb{R}^{n \times d}$, $y \in \mathbb{R}^n$ is

$$F(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

Its gradient is

$$\nabla F(\theta) = \frac{1}{n} X^\top (X\theta - y) = \frac{1}{n} X^\top X\theta - \frac{1}{n} X^\top y = H\theta - \frac{1}{n} X^\top y \quad \text{with} \quad H = \frac{1}{n} X^\top X.$$

Now apply gradient descent with a fixed step size $\gamma_t = \gamma$:

$$\theta_t = \theta_{t-1} - \gamma \nabla F(\theta_{t-1}) = \theta_{t-1} - \gamma \left(H\theta_{t-1} - \frac{1}{n} X^\top y \right).$$

Gradient Descent for OLS: error recursion

Note that the minimizer η^* satisfies

$$\nabla F(\eta^*) = H\eta^* - \frac{1}{n}X^\top y = 0 \quad \implies \quad H\eta^* = \frac{1}{n}X^\top y.$$

which implies

$$\nabla F(\theta) = H(\theta - \eta^*).$$

The gradient descent can be expressed as

$$\theta_t = \theta_{t-1} - \gamma H(\theta_{t-1} - \eta^*).$$

Subtracting η^* from both sides yields

$$\theta_t - \eta^* = \theta_{t-1} - \eta^* - \gamma H(\theta_{t-1} - \eta^*) = (I - \gamma H)(\theta_{t-1} - \eta^*).$$

Iterating this recursion gives

$$\theta_t - \eta^* = (I - \gamma H)^t(\theta_0 - \eta^*).$$

Gradient Descent for OLS: variable distance (1/5)

Substituting the explicit form of the iteration gives

$$\|\theta_t - \eta^*\|_2^2 = ((I - \gamma H)^t(\theta_0 - \eta^*))^\top ((I - \gamma H)^t(\theta_0 - \eta^*)).$$

Since H is symmetric, $I - \gamma H$ is also symmetric, so

$$\|\theta_t - \eta^*\|_2^2 = (\theta_0 - \eta^*)^\top (I - \gamma H)^{2t}(\theta_0 - \eta^*).$$

Lemma. Assume $0 < \mu = \lambda_{\min}(H) \leq \lambda_{\max}(H) = L$.

Then for every $x \in \mathbb{R}^d$,

$$\|(I - \gamma H)^t x\|_2^2 = x^\top (I - \gamma H)^{2t} x \leq \left(\max_{\lambda \in [\mu, L]} |1 - \gamma \lambda|^{2t} \right) \|x\|_2^2$$

Gradient Descent for OLS: variable distance (2/5)

Proof. Since H is symmetric, it is orthogonally diagonalizable:

$$H = U\Lambda U^\top,$$

where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$ with

$$\lambda_i \in [\mu, L].$$

Hence

$$I - \gamma H = U(I - \gamma \Lambda)U^\top,$$

and therefore

$$(I - \gamma H)^{2t} = U(I - \gamma \Lambda)^{2t}U^\top.$$

Gradient Descent for OLS: variable distance (3/5)

So the eigenvalues of $(I - \gamma H)^{2t}$ are

$$(1 - \gamma \lambda_i)^{2t}, \quad i = 1, \dots, d.$$

Thus

$$\lambda_{\max}((I - \gamma H)^{2t}) = \max_{\lambda \in [\mu, L]} |1 - \gamma \lambda|^{2t}.$$

Since $(I - \gamma H)^{2t}$ is symmetric,

$$\mathbf{x}^\top (I - \gamma H)^{2t} \mathbf{x} \leq \lambda_{\max}((I - \gamma H)^{2t}) \|\mathbf{x}\|_2^2.$$

This proves the result. □

Gradient Descent for OLS: variable distance (4/5)

Theorem. Choose $\gamma = \frac{1}{L}$ and denote $\kappa = \frac{L}{\mu}$. Then for gradient descent, we have

$$\|\theta_t - \eta^*\|_2^2 \leq \left(1 - \frac{1}{\kappa}\right)^{2t} \|\theta_0 - \eta^*\|_2^2 \leq e^{-2t/\kappa} \|\theta_0 - \eta^*\|_2^2.$$

Proof. From the error recursion,

$$\theta_t - \eta^* = (I - \gamma H)^t (\theta_0 - \eta^*).$$

Applying the lemma with $x = \theta_0 - \eta^*$ gives

$$\|\theta_t - \eta^*\|_2^2 \leq \left(\max_{\lambda \in [\mu, L]} |1 - \gamma \lambda|^{2t}\right) \|\theta_0 - \eta^*\|_2^2.$$

Now set $\gamma = 1/L$. For $\lambda \in [\mu, L]$,

$$0 \leq 1 - \frac{\lambda}{L} \leq 1 - \frac{\mu}{L} = 1 - \frac{1}{\kappa}.$$

Gradient Descent for OLS: variable distance (5/5)

Therefore,

$$\|\theta_t - \eta^*\|_2^2 \leq \left(1 - \frac{1}{\kappa}\right)^{2t} \|\theta_0 - \eta^*\|_2^2.$$

Finally, since $1 - x \leq e^{-x}$,

$$\left(1 - \frac{1}{\kappa}\right)^{2t} \leq e^{-2t/\kappa}.$$

We have

$$\|\theta_t - \eta^*\|_2^2 \leq \left(1 - \frac{1}{\kappa}\right)^{2t} \|\theta_0 - \eta^*\|_2^2 \leq e^{-2t/\kappa} \|\theta_0 - \eta^*\|_2^2.$$



Gradient Descent for OLS: function value (1/4)

Recall the OLS objective

$$F(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

Hence

$$F(\theta) - F(\eta^*) = \frac{1}{2n} \left(\|X\theta - y\|_2^2 - \|X\eta^* - y\|_2^2 \right).$$

Write

$$X\theta - y = X(\theta - \eta^*) + (X\eta^* - y).$$

Then

$$\|X\theta - y\|_2^2 = \|X(\theta - \eta^*)\|_2^2 + 2(\theta - \eta^*)^\top X^\top (X\eta^* - y) + \|X\eta^* - y\|_2^2.$$

Since the minimizer η^* satisfies

$$X^\top (X\eta^* - y) = 0,$$

the cross term vanishes. Therefore,

$$F(\theta) - F(\eta^*) = \frac{1}{2n} \|X(\theta - \eta^*)\|_2^2 = \frac{1}{2} (\theta - \eta^*)^\top H (\theta - \eta^*).$$

Gradient Descent for OLS: function value (2/4)

Replace θ by θ_t ,

$$F(\theta_t) - F(\eta^*) = \frac{1}{2}(\theta_t - \eta^*)^\top H(\theta_t - \eta^*).$$

Substituting again,

$$F(\theta_t) - F(\eta^*) = \frac{1}{2}((I - \gamma H)^t(\theta_0 - \eta^*))^\top H((I - \gamma H)^t(\theta_0 - \eta^*)).$$

Using symmetry and the fact that H commutes with $I - \gamma H$, we get

$$F(\theta_t) - F(\eta^*) = \frac{1}{2}(\theta_0 - \eta^*)^\top (I - \gamma H)^{2t} H(\theta_0 - \eta^*).$$

Gradient Descent for OLS: function value (3/4)

Theorem. Assume $0 < \gamma \leq \frac{1}{L}$. Then for gradient descent applied to OLS,

$$F(\theta_t) - F(\eta^*) \leq \frac{1}{8t\gamma} \|\theta_0 - \eta^*\|_2^2.$$

In particular, with $\gamma = \frac{1}{L}$, we have

$$F(\theta_t) - F(\eta^*) \leq \frac{L}{8t} \|\theta_0 - \eta^*\|_2^2.$$

Remark. This result does not require the minimizer to be unique; it still holds even when $\mu = 0$.

Gradient Descent for OLS: function value (4/4)

Proof. Recall that for OLS,

$$F(\theta_t) - F(\eta^*) = \frac{1}{2}(\theta_0 - \eta^*)^\top (I - \gamma H)^{2t} H (\theta_0 - \eta^*).$$

Like in the proof of convergence in distance, it is enough to bound

$$\lambda(1 - \gamma\lambda)^{2t}, \quad \lambda \in [0, L].$$

Using elementary calculus, we know that

$$\lambda(1 - \gamma\lambda)^{2t} \leq \frac{1}{4t\gamma}.$$

Therefore,

$$F(\theta_t) - F(\eta^*) \leq \frac{1}{2} \cdot \frac{1}{4t\gamma} \|\theta_0 - \eta^*\|_2^2 = \frac{1}{8t\gamma} \|\theta_0 - \eta^*\|_2^2.$$

Solving OLS with Gradient Descent in Python

```
import numpy as np
# X : (n, d) design matrix; y : (n,) response vector
# theta0 : (d,) initial guess; gamma : step size
# tol : tolerance; max_iter : maximum number of iterations
theta = theta0.copy(); k = 0
while k < max_iter:
    # Step 1: compute the gradient
    grad = X.T @ (X @ theta - y)
    # Step 2: stopping criterion
    if np.linalg.norm(grad) < tol: break
    # Step 3: gradient descent update
    theta = theta - gamma * grad
    k += 1
```

This returns an approximate least-squares solution.

Outline

Least Squares and QR decomposition

Least Squares via First Order Method

Least Squares via Second Order Method

Newton's method

The second-order idea is to approximate the objective near the current iterate by a quadratic model.

For a twice differentiable function,

$$F(\theta) \approx F(\theta_{t-1}) + \nabla F(\theta_{t-1})^\top (\theta - \theta_{t-1}) + \frac{1}{2}(\theta - \theta_{t-1})^\top \nabla^2 F(\theta_{t-1})(\theta - \theta_{t-1}).$$

Minimizing this local quadratic approximation gives the Newton update:

$$\theta_t = \theta_{t-1} - [\nabla^2 F(\theta_{t-1})]^{-1} \nabla F(\theta_{t-1}).$$

Intuition: Newton uses curvature information to rescale the gradient.

Newton method for least squares

For least squares,

$$F(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2,$$

with

$$\nabla F(\theta) = \frac{1}{n} X^\top (X\theta - y), \quad \nabla^2 F(\theta) = \frac{1}{n} X^\top X.$$

Since the Hessian is constant, Newton's method gives

$$\theta_t = \theta_{t-1} - \left(\frac{1}{n} X^\top X \right)^{-1} \left(\frac{1}{n} X^\top (X\theta_{t-1} - y) \right).$$

After simplification,

$$\theta_t = (X^\top X)^{-1} X^\top y = \hat{\theta}.$$

Conclusion: for quadratic least squares, Newton reaches the exact minimizer in one step.

Solving OLS with Newton's Method in Python

```
import numpy as np
# Input:
# X : (n, d) design matrix; y : (n,) response vector
# theta0 : (d,) initial guess
theta = theta0.copy()
# Step 1: compute the gradient at theta0
grad = X.T @ (X @ theta - y)
# Step 2: form the Hessian
H = X.T @ X
# Step 3: solve H p = grad
p = np.linalg.solve(H, grad)
# Step 4: Newton update
theta = theta - p
```

For OLS, Newton's method reaches the solution in one step when $X^T X$ is invertible.

Main messages

1. **Direct methods:** Least squares admits a **closed-form** solution:

$$\hat{\theta} = (X^T X)^{-1} X^T y.$$

and QR decomposition gives a stable numerical implementation.

2. **First-order methods:** computing the gradient. It is **iterative**, **cheap** per iteration, and its speed depends on the condition number.
3. **Second-order methods:** computing the gradient and inverse Hessian. For quadratic least squares, it finds the minimizer in **one step**.